

Advanced Cryptography

Elisabeth Oswald

Differential Input Attacks

ROADMAP

Conceptual View

Single vs Differential Attacks

Differential Input Attacks

Five step program

Example

SIDE CHANNELS

It is perhaps pretty obvious that data dependent code optimisations lead to data dependent execution times. It is way less obvious to write “constant time code”.

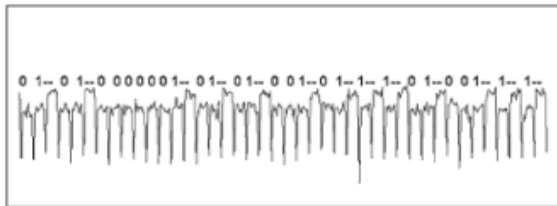
Perhaps even more intriguing is the fact that the amount of instantaneous power that a circuit consumes, or the type of electromagnetic emanations that a circuit produces, is strongly instruction and data dependent. It is non-trivial to avoid this.

Therefore “getting rid” of timing, power, and EM side channels does not seem to be a realistic option at the moment. This is our motivation to look at implementation strategies that make them “provably harder”.

SIDE CHANNEL ATTACKS: SINGLE TRACE

Scenario: When utilising side channel information the adversary typically also has access to inputs/outputs of the implementation.

If the data dependency dominates what can be measured, then it is possible that a **single observation** of the action of a cryptographic process that uses a secret key reveals this key. Such an unfortunate situation can occur in practice, see an exemplary power trace of an RSA decryption below:



AES

```
1 AES_Encrypt (ByteArray m, ByteArray k)
2   ByteArray c
3   state = m
4   rk = KeyExpansion(k)
5   AddRoundKey(state,  $rk_0$ )
6   for r = 1 step 1 to 9
7       SubBytes(state)
8       ShiftRows(state)
9       MixColumns(state)
10      AddRoundKey(state,  $rk_r$ )
11   end
12   SubBytes(state)
13   ShiftRows(state)
14   AddRoundKey(state,  $rk_{10}$ )
15   c = state;
```

SINGLE TRACE: AES

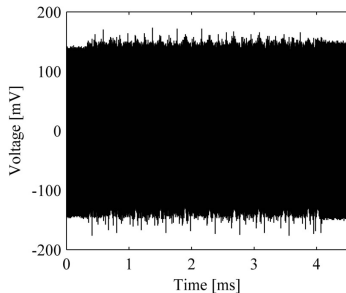


Figure: Full AES encryption.

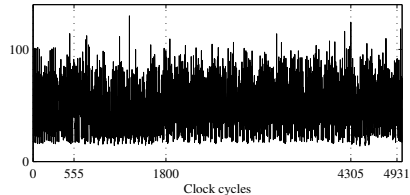


Figure: One round of AES

In case of an algorithm like AES, there is no simple association between “flow” and secret key bits. Therefore we can gather useful information by looking/analysing a single trace, but unless some sophisticated power models are available, we cannot extract the key as such.

SIDE CHANNEL ATTACKS: DIFFERENTIAL INPUT ATTACKS

Especially in the case of implementations that come **with countermeasures**, a single observation will not suffice to extract the secret key.

When this is the case, the adversary will obviously try to gather more traces.

It may well make sense to try and gather data repeating the cryptographic process with the same input data, but more often it is useful to perform the cryptographic process with **varying input data**.

The situation where an adversary can observe varying inputs is called a **differential input attack**. In many publications you will then find the name **Differential Power Analysis** (DPA), differential timing analysis (DTA), differential EM analysis (DEMA), etc.

ROADMAP

Conceptual View

Single vs Differential Attacks

Differential Input Attacks

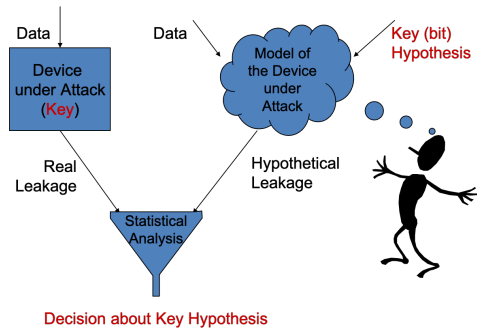
Five step program

Example

DIFFERENTIAL ATTACKS

The right hand picture illustrates the principle of all DPA style attacks.

I use the term “DPA style” or “differential style” even in the context of timing and EM attacks.

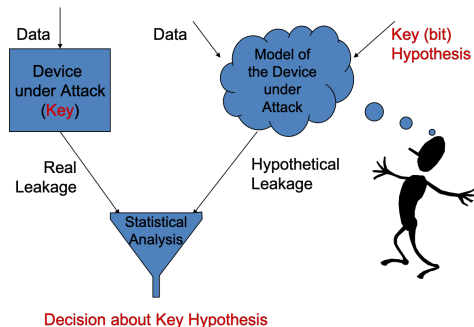


DIFFERENTIAL ATTACKS

DPA style or differential attacks work whenever we have a static secret that is used by crypto function with (at least partially) known inputs or outputs.

Depending on the type of scheme, the device and the implementation strategy, one can often make guesses about just a few bits of the key at a time: i.e. we use a **divide and conquer** strategy.

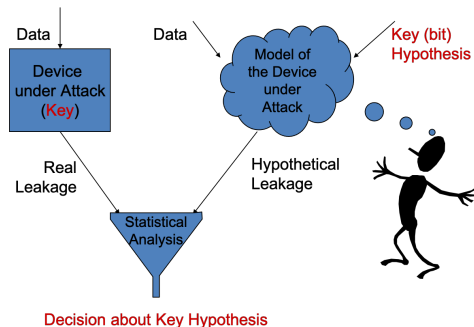
Because of the statistical analysis, it is possible to exploit even the “tiniest” of leakage.



DIFFERENTIAL ATTACKS

There are a number of choices that are necessary of “configuring” a concrete attack:

- ▶ The model: intermediate value and leakage model
- ▶ The measurements: what side channel and how/what to measure
- ▶ The statistical analysis



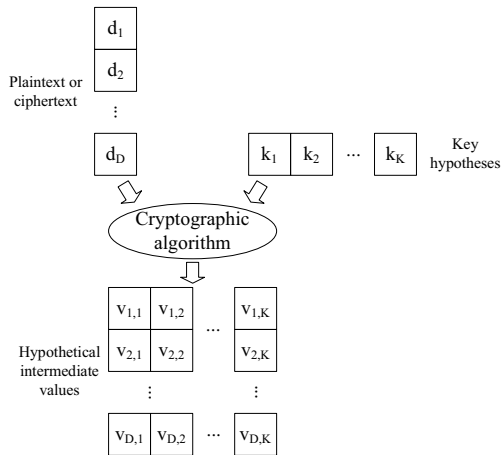
In the notes I explain five distinctive steps/choices, which we now run through.

DIFFERENTIAL ATTACKS: INTERMEDIATE VALUE

1. Choose an intermediate value that depends on only a small part of the input and the unknown key.

AES: single state byte after AddRoundKey, SubBytes, ShiftRows, or, two or more state bytes within/after MixColumns.

3. Computing all combinations of key guesses and inputs, using the crypto algorithm's steps, then gives predicted/hypothetical intermediate values.

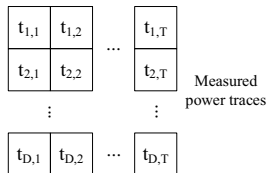


DIFFERENTIAL ATTACKS: RECORD TRACES

The choice of intermediate value will probably guide how/what to measure.

If plaintexts are available, and a value in the first round is selected, record a time period at the start of encryption.

2. For each input, we record a trace.

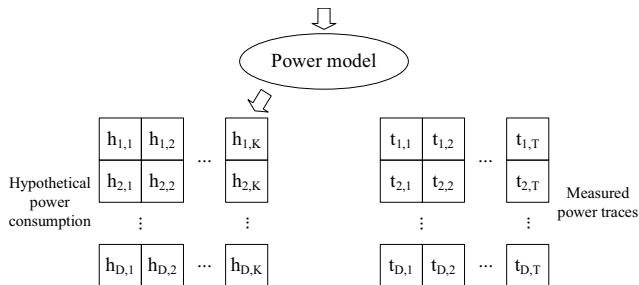


DIFFERENTIAL ATTACKS: HYPOTHETICAL LEAKAGES

We already have, for each input, all possible hypothetical intermediate values.

We now need to decide how to “interpret” these for the purpose of statistical inference: how do the predicted values relate to the actual observations?

4. We use an adversarial leakage model (in the case of power analysis, we call it a power model) for this purpose.

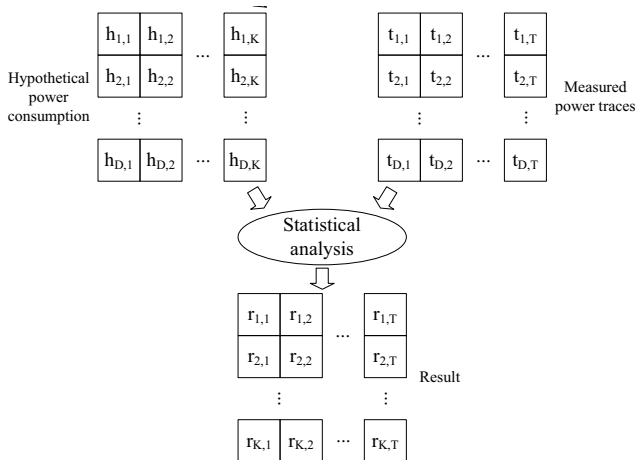


DIFFERENTIAL ATTACKS: HYPOTHETICAL LEAKAGES

For each input, we have hypothetical leakage values and a side channel trace.

We know wish to determine which of the “columns” in our hypothetical values indicates the value of the correct key.

We do this by using statistics: each column is “compared” with all trace points.



ROADMAP

Conceptual View

Single vs Differential Attacks

Differential Input Attacks

Five step program

Example

AES IMPLEMENTATION

We target an implementation of AES on a typical 8-bit microprocessor. This means that the databus can transfer 8 bits at a time, and all operations like XOR, AND, $\dots$, that the ALU implements work on 8 bit values.

We target the SubBytes output of AES in round 1.

We know that the implementation for SubBytes is based on a stored table: i.e. the SubBytes output will be transferred over the databus. Databus leakage is often strongly related to the Hamming weight of the data.

We configure our equipment so that it records just the beginning of the encryption operation. We store 1000 traces as variable/matrix T , and the plaintext input.

AES IMPLEMENTATION, CONT.

We only need to use the first byte of the 16 byte plaintext to guess the output of the first SubBytes. Using our 1000 inputs, and a single byte key guess, we calculate the values $v_{i,j} = S(d_i \oplus k_j)$, where $d_1, \dots, d_{1000}$ are the first byte of each of the 1000 plaintexts and $k_j = j - 1$ with $j = 1, \dots, 256$.

The matrix $\mathbf{V}$ has hence a size of 1000×256 values (step 3).

Next we map this matrix to hypothetical power values using the Hamming weight leakage model, leading to a matrix $\mathbf{H}$, $h_{i,j} = HW(v_{i,j})$.

We use the correlation coefficient to determine the linear relationship between the columns $\mathbf{h}_i$ and $\mathbf{t}_j$ for $i = 1, \dots, K$ and $j = 1, \dots, T$.

AES IMPLEMENTATION, CONT.

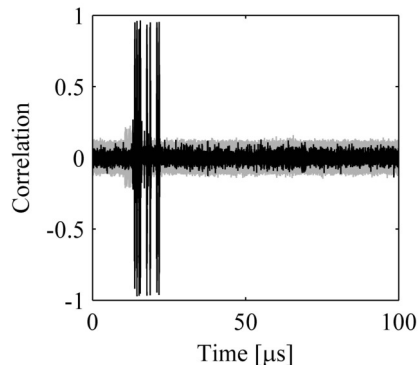


Figure: Correct key hypothesis 225 is plotted in black, while all other key hypotheses are plotted in grey.

The result gives the column 225 the highest correlation values. This indicates that the first byte of the secret key has the value 225.

AES IMPLEMENTATION, CONT.

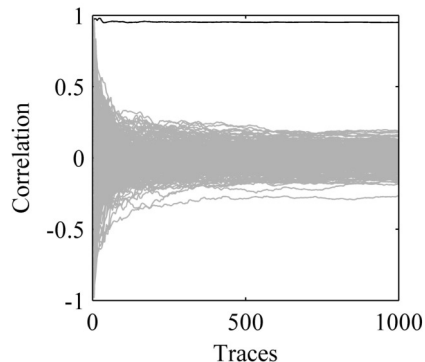


Figure: Correct key hypothesis 225 is plotted in black, while all other key hypotheses are plotted in grey.

This plot indicates that by far fewer traces would suffice for identifying this first byte.